

WEEK2 ASSIGNMENT3

AI-powered Task Management App / Backend AI Engineering project

Prompt Iteration Log

Task:

Design backend API + AI prioritization feature for my task management app.

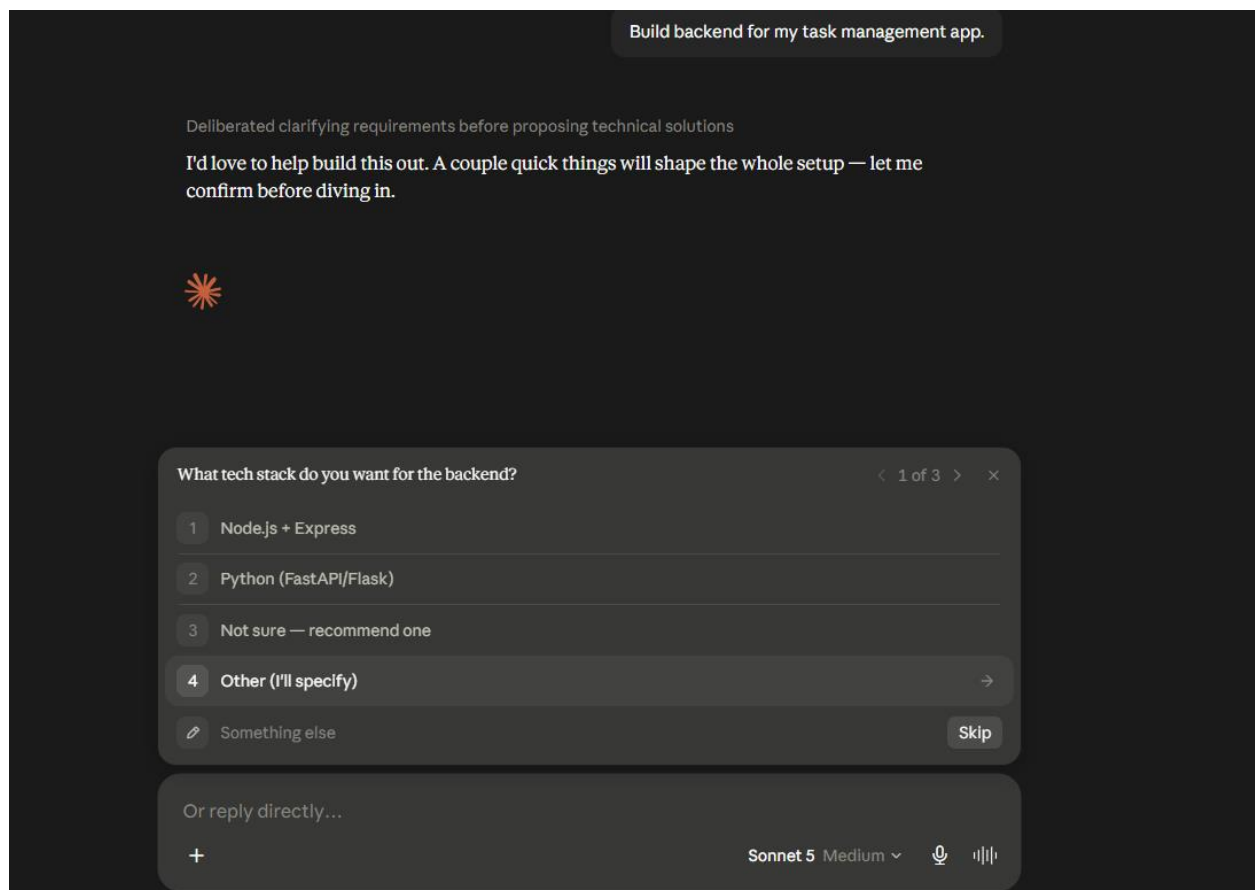
Version 0 → Naive Prompt

Technique: None (baseline)

Prompt:

Build backend for my task management app.

Claude snaps



Notes:

No changes made

Version 1 — Role Assignment

Technique added: Role assignment

Prompt:

Act as a senior backend AI engineer. Build the backend for my task management app.

Architected full-stack solution with Node.js, Express, and SQLite authentication

Got it — I'll make the calls: **Node.js + Express, SQLite, JWT auth**. This is a solid, dependency-light stack that's easy to run locally or deploy anywhere. Building a full REST API for tasks with user accounts.

Running command

Notes:

- Change: Added expert role.
- Improvement: Output had more technical suggestions and better architecture decisions.
- Still failed: Did not know project context.
- Next: Add context.

Version 2 — Context and Motivation

Technique added: Context and motivation

Prompt:

Act as a senior backend AI engineer.

I am a Software Engineering student building a portfolio project. The app is for university students and interns who need help managing tasks, deadlines, and priorities.

Design the backend for this AI-powered task management app.

1. Tech Stack

Layer	Choice	Why
Runtime	Node.js + Express	Fast to build, huge ecosystem, easy to demo/deploy for a portfolio
Database	PostgreSQL	Relational data (users → tasks → categories) with real constraints; free tier on Supabase/Railway/Neon
ORM	Prisma	Type-safe queries, migrations, great for showcasing clean schema design
Auth	JWT (access + refresh tokens)	Stateless, standard for portfolio-grade APIs
AI layer	Anthropic API (Claude) via a dedicated service module	Powers prioritization, deadline extraction, smart suggestions
Validation	Zod	Schema validation at the request boundary
Deployment	Render / Railway (API) + Neon/Supabase (Postgres)	Free tiers, good for demoing in a portfolio

2. Core Data Model

```
User
├─ id, name, email, password_hash, timezone, created_at
└─
Task
├─ id, user_id (FK), title, description
├─ status: todo | in_progress | done
├─ priority: low | medium | high | urgent
├─ priority_source: manual | ai_suggested
├─ due_date, estimated_minutes
├─ category_id (FK, nullable)
├─ created_at, updated_at
└─
Category
├─ id, user_id (FK), name, color // e.g. "CS401", "Internship - Acme", "Personal"
└─
AiInsight
├─ id, user_id (FK), task_id (FK, nullable)
├─ type: priority_suggestion | deadline_risk | daily_plan
├─ content (text/JSON), created_at
```

Why a separate `AiInsight` table rather than bolting AI output onto `Task`: it lets you log every AI suggestion, show "why the AI suggests this," and build a history/analytics view later — good portfolio talking point (auditable AI decisions).

3. API Endpoints

Auth

- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/refresh`

Tasks

- `GET /api/tasks` — filter by status, category, due date range
- `POST /api/tasks`
- `GET /api/tasks/:id`
- `PATCH /api/tasks/:id`
- `DELETE /api/tasks/:id`

Categories

- `GET /api/categories`
- `POST /api/categories`

AI features (the differentiator)

- `POST /api/ai/prioritize` — sends the user's open tasks to Claude, returns a re-ranked priority list with reasoning
- `POST /api/ai/parse-task` — user pastes free text ("Midterm project due next Friday, worth 30%") → AI extracts title, due date,  rity
- `GET /api/ai/daily-plan` — generates a suggested "today" plan given tasks + deadlines + estimated effort

4. AI Integration Pattern

Keep AI calls in an isolated service layer, never directly in route handlers:

```
src/services/aiService.js
- buildPrioritizationPrompt(tasks)
- callClaude(prompt)
- parseStructuredResponse(response) // enforce JSON output from Claude
```

This matters for two reasons worth calling out in a portfolio writeup:

- **Testability** — you can mock `aiService` in tests without hitting the real API.
- **Reliability** — force Claude to respond in strict JSON (via prompt + `response_format` discipline) so the rest of the app never has to parse prose.

5. Architecture Diagram

Want me to generate a visual diagram of this (request/response flow through auth → routes → services → DB/AI), or move straight into scaffolding the actual code?

Notes:

- Improvement: Suggestions became more relevant to student users and portfolio scope.
- Still failed: Response was not structured enough.

Version 3 — Few-shot Examples

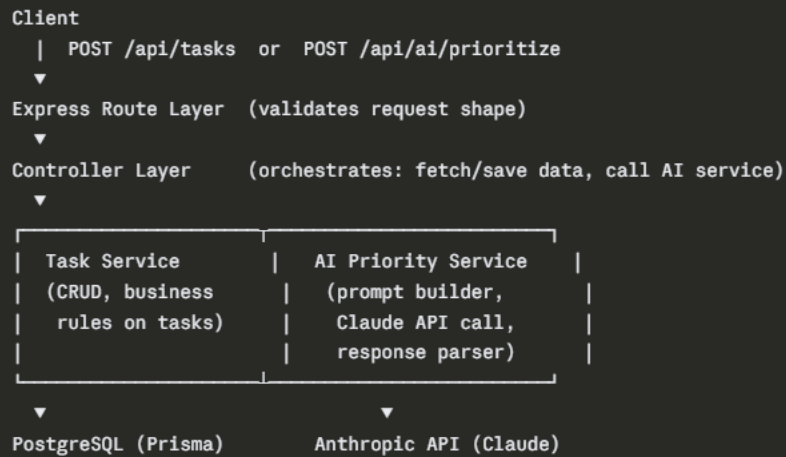
Technique added: Few-shot examples

Prompt:

Notes:

- Improvement: AI understood the expected AI feature behavior better.
- Still failed: Architecture explanation was mixed and difficult to implement.

1. High-Level Flow



2. Data Model

User

- id, name, email, password_hash, timezone

Task

- id, user_id (FK)
- title, description
- deadline (datetime)

↓

2. Data Model

User

└ id, name, email, password_hash, timezone

Task

└ id, user_id (FK)

└ title, description

└ deadline (datetime)

└ status: pending | in_progress | done

└ priority: high | medium | low | null // set by AI or manually

└ priority_reason: text | null // AI-generated explanation

└ priority_source: ai | manual | null

└ created_at, updated_at

AIPriorityLog

// audit trail of AI decisions

└ id, task_id (FK), user_id (FK)

└ input_snapshot (JSON) // title, deadline, status at time of call

└ output_priority, output_reason

└ model_used, created_at

3. Request → AI → Response Flow (the example feature)

1. Client sends task:

```
{ title: "Complete assignment", deadline: "2026-08-07", status: "pending" }
```

2. Controller (POST /api/tasks or /api/ai/prioritize):

a. Save/fetch task row

b. Call `AIPriorityService.evaluate(task)`

3. AIPriorityService:

a. `buildPrompt(task)` → structured prompt instructing Claude to return strict JSON: { "priority": "...", "reason": "..." }

b. `callClaude(prompt)` → Anthropic /v1/messages

c. `parseAndValidate(response)` → guard against malformed output (zod schema: priority ∈ {high, medium, low}, reason: string)

4. Controller:

a. Update task row: priority, priority_reason, priority_source = 'ai'

b. Insert row into `AIPriorityLog` (audit trail)

c. Return response to client:

```
{
  "task": "Complete assignment",
  "priority": "High",
  "reason": "Close deadline and important academic impact."
}
```

4. Prompt Contract (AiPriorityService)

Claude is called with a system prompt that pins the output shape and reasoning criteria, so the service layer never has to parse free text:

System: You are a task prioritization engine. Given a task's title, deadline, and status, output ONLY valid JSON:

```
{"priority": "High|Medium|Low", "reason": "<one short sentence>"}
```

Criteria: urgency (time to deadline), academic/work impact, current status.

5. Module Layout

```
src/
├─ routes/
│   ├─ tasks.routes.js
│   └─ ai.routes.js
├─ controllers/
│   ├─ tasks.controller.js
│   └─ ai.controller.js
├─ services/
│   ├─ task.service.js
│   └─ aiPriority.service.js    // prompt build, Claude call, validation
├─ schemas/
│   ├─ task.schema.js         // zod: request validation
│   └─ aiResponse.schema.js   // zod: validate Claude's JSON output
├─ middleware/
│   ├─ auth.js
│   └─ errorHandler.js
└─ db/
    └─ prisma.schema
```

6. Key Design Decisions

Decision	Reason
Separate <code>AiPriorityService</code> from <code>TaskService</code>	AI logic is swappable/mockable; task CRUD doesn't depend on AI being available
<code>AiPriorityLog</code> table	Every AI decision is auditable — traceable "why" for each priority
Strict JSON contract enforced via schema validation	Prevents AI prose from leaking into the app; fails safely if Claude's output drifts
<code>priority_source</code> field on Task	Distinguishes AI-assigned vs. user-overridden priority — useful for showing AI accuracy over time
Async fallback: if AI call fails/times out	Task still saves with <code>priority: null</code> ; client can retry prioritization independently

Ready to scaffold this into actual code when you want — routes, Prisma schema, and the AI service with the prompt above.

Version 4 — Output Structure

Technique added: Output structure

Prompt:

Act as a senior backend AI engineer.

I am building an AI-powered task management app for students and interns.

Provide the answer using this structure:

1. Technology stack
2. Database schema
3. API endpoints
4. AI integration flow
5. Security considerations
6. Testing approach

Notes:

- Improvement: Output became organized and easier to follow.
- Still failed: Some steps were too broad.

1. Technology Stack

Layer	Choice
Runtime	Node.js + Express
Database	PostgreSQL
ORM	Prisma
Auth	JWT (access + refresh tokens), bcrypt for password hashing
Validation	Zod
AI Provider	Anthropic API (Claude)
Testing	Jest + Supertest
Deployment	Render/Railway (API), Neon/Supabase (Postgres)
Logging	pino or winston

2. Database Schema

User

- └ id (PK)
- └ name
- └ email (unique)
- └ password_hash
- └ timezone

↓

```

└─ created_at

Category
└─ id (PK)
└─ user_id (FK → User)
└─ name // e.g. "CS401", "Internship - Acme"
└─ color

Task
└─ id (PK)
└─ user_id (FK → User)
└─ category_id (FK → Category, nullable)
└─ title
└─ description
└─ deadline (datetime)
└─ status: pending | in_progress | done
└─ priority: high | medium | low | null
└─ priority_reason: text | null
└─ priority_source: ai | manual | null
└─ estimated_minutes
└─ created_at, updated_at

AiPriorityLog
└─ id (PK)
└─ task_id (FK → Task)
└─ user_id (FK → User)
└─ input_snapshot (JSON)
└─ output_priority
└─ output_reason
└─ model_used
└─ created_at

RefreshToken
└─ id (PK)
└─ user_id (FK → User)
└─ token_hash
└─ expires_at
└─ revoked (boolean)

```

3. API Endpoints

Auth

- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/refresh`
- `POST /api/auth/logout`

Tasks

- `GET /api/tasks` — filters: status, category, deadline range
- `POST /api/tasks`
- `GET /api/tasks/:id`
- `PATCH /api/tasks/:id`
- `DELETE /api/tasks/:id`

Categories

- `GET /api/categories`
- `POST /api/categories`
- `DELETE /api/categories/:id`

AI

- `POST /api/ai/prioritize/:taskId` — evaluate a single task
- `POST /api/ai/prioritize-batch` — evaluate all open tasks
- `POST /api/ai/parse-task` — free text → structured task (title, deadline, priority)
- `GET /api/ai/daily-plan` — suggested task order for today

4. AI Integration Flow

1. Client submits task (title, deadline, status)
2. Controller saves/fetches task, calls `aiPriorityService.evaluate(task)`
3. `aiPriorityService`:
 - a. `buildPrompt(task)` — system prompt fixes output to strict JSON: `{ "priority": "High|Medium|Low", "reason": "<one sentence>" }`
 - b. `callClaude(prompt)` → Anthropic `/v1/messages`
 - c. `validate(response)` via Zod — reject/retry on malformed output
4. Controller updates Task (priority, priority_reason, priority_source = 'ai') and inserts an `aiPriorityLog` row for auditability
5. Response returned to client:

```
{ "priority": "High", "reason": "Close deadline and important academic impact." }
```
6. On AI failure/timeout: task saves with priority: null; client can retry prioritization independently without blocking task creation

5. Security Considerations

- Hash passwords with bcrypt (cost factor ≥ 12); never store plaintext.
- Short-lived JWT access tokens (~15 min) + rotating refresh tokens stored hashed in DB.
- All task/category routes scoped to `req.userId` from JWT — never trust client-supplied user IDs.
- Rate-limit `/api/ai/*` routes separately (AI calls are costlier and abusable).
- Validate and sanitize all input with Zod before it reaches the DB or the AI prompt (prevents prompt injection via task titles/descriptions).
- Enforce strict JSON-only output from Claude and validate against a schema before persisting — never execute or directly render raw AI text.
- Use HTTPS only; set secure, httpOnly cookies if refresh tokens are cookie-based.
- Environment secrets (JWT secret, Anthropic API key) via `.env`, never committed; use a secrets manager in production.
- CORS restricted to known frontend origins.
- Log AI inputs/outputs (`aiPriorityLog`) without storing sensitive personal data beyond what's needed for audit.

6. Testing Approach

- **Unit tests** (Jest): service-layer logic — `TaskService`, `aiPriorityService` prompt builder and response validator — with the Anthropic API mocked.
- **Integration tests** (Supertest): full request/response cycle for auth and task CRUD against a test Postgres DB (Dockerized or in-memory via testcontainers).
- **Contract tests for AI output**: feed the validator known-good and known-bad Claude responses to confirm malformed JSON is rejected, not silently accepted.
- **Auth tests**: token expiry, refresh rotation, access-control checks (user A cannot access user B's tasks).
- **Error-path tests**: AI timeout/failure → task still persists with `priority: null`.
- CI pipeline (GitHub Actions): run lint + unit + Integration tests on every PR against a throwaway test database.

Version 5 — Step Decomposition

Technique added: Step decomposition

Prompt:

Act as a senior backend AI engineer.

I am building an AI-powered task management app for students and interns.

Help me design it step by step.

Step 1: Recommend technology stack.

Step 2: Design database models.

Step 3: Create API endpoints.

Step 4: Explain LLM integration.

Step 5: Explain testing strategy.

Wait for my confirmation after each step.

Notes:

- Improvement: AI responses became more focused and easier to review.
- Still failed: Required multiple interactions.

Step 1: Technology Stack		
Layer	Choice	Why
Runtime	Node.js + Express	Fast to build, widely understood, easy to demo
Database	PostgreSQL	Relational integrity for users → tasks → categories
ORM	Prisma	Type-safe queries, clean migrations, good for a portfolio walkthrough
Auth	JWT (access + refresh tokens) + bcrypt	Stateless, industry-standard for API portfolios
Validation	Zod	Enforces request shape and validates AI JSON output
AI Provider	Anthropic API (Claude)	Powers prioritization, deadline parsing, daily planning
Testing	Jest + Supertest	Unit + Integration coverage
Deployment	Render/Railway (API) + Neon/Supabase (Postgres)	Free tiers, easy to demo live
Confirm this stack, or tell me what to swap (e.g. different DB, different auth approach) before we move to Step 2 (database models).		

Step 2: Database Models

```
User
├─ id (PK)
├─ name
├─ email (unique)
├─ password_hash
├─ timezone
├─ created_at

Category
├─ id (PK)
├─ user_id (FK → User)
├─ name // e.g. "CS401", "Internship - Acme"
├─ color

Task
├─ id (PK)
├─ user_id (FK → User)
├─ category_id (FK → Category, nullable)
├─ title
├─ description
├─ deadline (datetime)
├─ status: pending | in_progress | done
├─ priority: high | medium | low | null
├─ priority_reason: text | null
├─ priority_source: ai | manual | null
├─ estimated_minutes
├─ created_at, updated_at

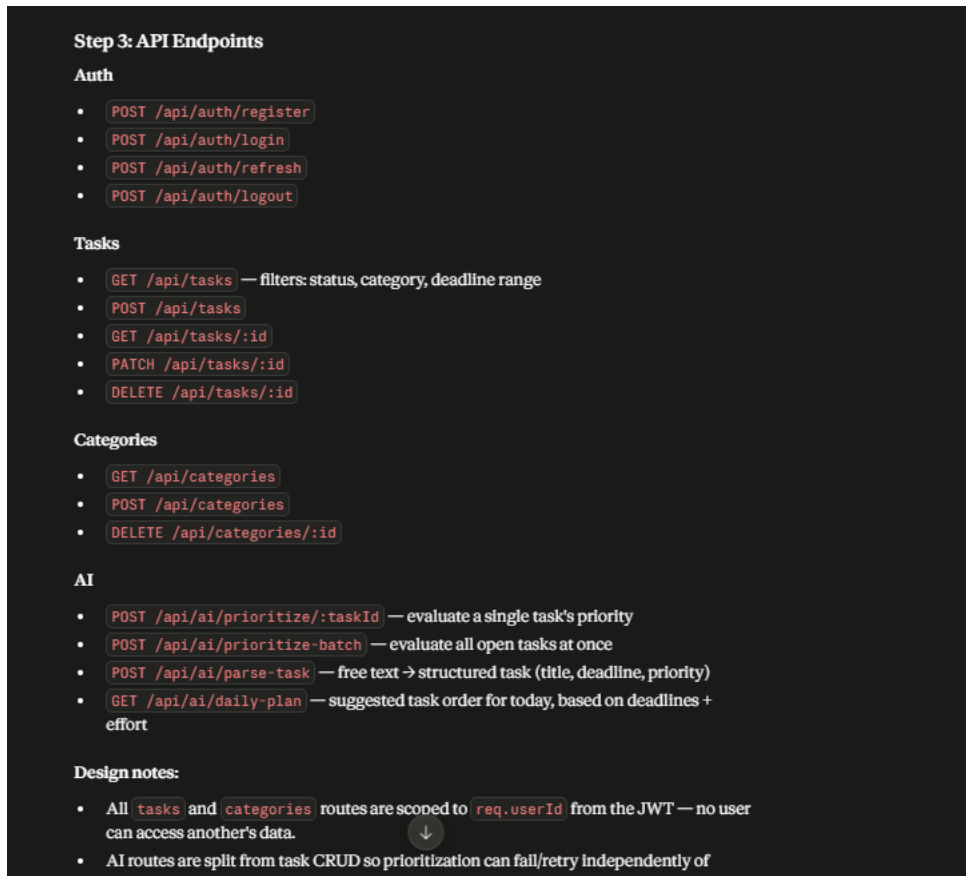
AiPriorityLog // audit trail of every AI decision
├─ id (PK)
├─ task_id (FK → Task)
├─ user_id (FK → User)
├─ input_snapshot (JSON) // title, deadline, status at call time
├─ output_priority
├─ output_reason
├─ model_used
├─ created_at

RefreshToken
├─ id (PK)
├─ user_id (FK → User)
├─ token_hash
├─ expires_at
├─ revoked (boolean)
```

```
├─ created_at, updated_at

AiPriorityLog // audit trail of every AI decision
├─ id (PK)
├─ task_id (FK → Task)
├─ user_id (FK → User)
├─ input_snapshot (JSON) // title, deadline, status at call time
├─ output_priority
├─ output_reason
├─ model_used
├─ created_at

RefreshToken
├─ id (PK)
├─ user_id (FK → User)
├─ token_hash
├─ expires_at
├─ revoked (boolean)
```



Act as a senior backend AI engineer.

I am working on an AI-powered task management application for university students and interns.

The goal is to design a backend system that helps users manage tasks and receive AI-based priority suggestions using LLM capabilities.

The users are university students and interns balancing coursework, internships, and personal responsibilities.

Here is an example of the expected quality:

For a task like "Submit database assignment tomorrow", the AI should identify it as high priority and explain the reasoning based on urgency, impact, and effort.

Complete the task using this structure:

1. Technology stack recommendation
2. Database schema design
3. REST API endpoints
4. AI integration workflow
5. Error handling approach
6. Testing strategy

Break the solution into clear steps:

First explain the architecture, then database design, then API implementation, and finally AI integration.

Consider:

- Keep the architecture realistic for a student portfolio project.
- Avoid unnecessary complexity.
- Explain technical decisions clearly.
- Focus on maintainable and scalable design.

Architecture Overview

A layered Express API: routes validate input → controllers orchestrate → services hold business logic (task management separate from AI logic) → Prisma/Postgres for persistence, Anthropic API for reasoning. This separation keeps the AI provider swappable and testable without touching core task logic — realistic scope for a portfolio project, no microservices or queues needed at this stage.

```
Client → Routes → Controllers → Services (Task | AI) → DB / Claude API
```

1. Technology Stack

Layer	Choice	Why
Runtime	Node.js + Express	Fast to build and demo, widely understood
Database	PostgreSQL	Relational integrity across users/tasks/categories
ORM	Prisma	Type-safe queries, clean migrations
Auth	JWT (access + refresh) + bcrypt	Stateless, standard for API portfollos
Validation	Zod	Validates requests and AI JSON output
AI Provider	Anthropic API (Claude)	Priority reasoning, deadline parsing
Testing	Jest + Supertest	Unit + integration coverage
Deployment	Render/Railway + Neon/Supabase	Free tiers, easy live demo

2. Database Schema

```
User
├─ id, name, email (unique), password_hash, timezone, created_at

Category
├─ id, user_id (FK), name, color

Task
├─ id, user_id (FK), category_id (FK, nullable)
├─ title, description, deadline
├─ status: pending | in_progress | done
```

3. REST API Endpoints

Auth: `POST /api/auth/register`, `login`, `refresh`, `logout`

Tasks: `GET/POST /api/tasks`, `GET/PATCH/DELETE /api/tasks/:id` (filters: status, category, deadline range)

Categories: `GET/POST /api/categories`, `DELETE /api/categories/:id`

AI:

- `POST /api/ai/prioritize/:taskId` — evaluate one task
- `POST /api/ai/prioritize-batch` — evaluate all open tasks
- `POST /api/ai/parse-task` — free text → structured task
- `GET /api/ai/daily-plan` — suggested order for today

All task/category routes scoped to `req.userId` from the JWT.

4. AI Integration Workflow

1. Client sends task: { title: "Submit database assignment", deadline: "tomorrow", status: "pending" }
2. Controller saves/fetches task → calls `AiPriorityService.evaluate(task)`
3. `AiPriorityService`:
 - a. `buildPrompt(task)` — system prompt instructs Claude to weigh urgency (time to deadline), impact (academic/work weight), and effort (estimated_minutes if present), returning strict JSON: { "priority": "High|Medium|Low", "reason": "<one sentence>" }
 - b. `callClaude(prompt)` → Anthropic `/v1/messages`
 - c. `validate(response)` via Zod — reject malformed/off-schema output
4. Controller updates Task (priority, priority_reason, priority_source='ai'), inserts `AiPriorityLog` row for audit trail
5. Response: { "priority": "High", "reason": "Due tomorrow with direct academic impact and moderate remaining effort." }

Example output matches the given case: near-term deadline + academic weight → High, with reasoning grounded in the same three factors every time (consistent prompt contract = consistent reasoning quality).

5. Error Handling Approach

- **AI call fails/times out:** task still saves with `priority: null`; return 202-style response indicating prioritization can be retried, never block task creation on AI availability.
- **Malformed AI output:** Zod validation rejects it; retry once with a stricter reminder prompt, then fall back to `priority: null` + log the raw response for debugging.
- **DB errors:** centralized error-handling middleware maps known Prisma errors (unique constraint, foreign key) to clean 4xx responses; unexpected errors → generic 500, logged with request context.
- **Auth errors:** expired/invalid JWT → 401; user attempting to access another user's task → 403/404 (don't leak existence of the resource).
- **Input validation errors:** Zod schema failures return 400 with field-level messages before any DB or AI call is made.

6. Testing Strategy

- **Unit tests:** `TaskService` and `AiPriorityService` (prompt builder + response validator) with the Anthropic API mocked — covers logic without real API cost/latency.
- **Integration tests:** full request/response cycles for auth and task CRUD against a test Postgres instance (Docker or testcontainers).
- **Contract tests:** feed the AI response validator known-good and known-bad JSON payloads to confirm malformed output is always rejected, never persisted.
- **Auth tests:** token expiry, refresh rotation, cross-user access denial.
- **Error-path tests:** AI timeout → task persists with `priority: null`; retry logic triggers correctly.
- **CI (GitHub Actions):** lint + unit + integration tests on every PR against a disposable test database.

Chagpt snaps:

Sample Output (Final Prompt Run)

1. Technology Stack Recommendation

Backend: Node.js + Express.js

Database: PostgreSQL

ORM: Prisma

Authentication: JWT-based authentication

AI Integration: OpenAI/Claude API for task prioritization suggestions

Reasoning:

This stack is suitable for a student portfolio project because it is widely used, has strong community support, and allows clear separation between frontend, backend, database, and AI services.

2. Database Schema Design

User Table

- id
- name
- email
- password_hash
- created_at

Task Table

- id
- user_id
- title
- description
- due_date
- status
- priority
- created_at

AI_Priority_Result Table

- id
- task_id
- suggested_priority
- reasoning
- created_at

3. REST API Endpoints

Authentication

POST /api/auth/register

Create a new user account.

POST /api/auth/login

Authenticate user and return token.

Tasks

GET /api/tasks

Retrieve user tasks.

POST /api/tasks

Create a new task.

PUT /api/tasks/:id

Update task details.

DELETE /api/tasks/:id

Delete a task.

PATCH /api/tasks/:id/complete

Mark task as completed.

AI Priority

POST /api/tasks/:id/prioritize



4. AI Integration Workflow

Step 1:

User creates or selects a task.

Step 2:

Backend collects task information:

- title
- description
- deadline
- completion status

Step 3:

Backend sends structured context to the LLM.

Step 4:

LLM evaluates:

- urgency
- impact
- effort

Step 5:

AI response is validated and stored.

Step 6:

Frontend displays priority level and reasoning.



5. Error Handling Approach

- Validate user input before processing.
- Handle failed AI API requests gracefully.
- Allow users to continue managing tasks if AI suggestions are unavailable.
- Log errors for debugging.
- Avoid making the application completely dependent on AI availability.

6. Testing Strategy

Backend Testing

- Test authentication flow.
- Test CRUD operations.
- Test API error responses.

AI Feature Testing

- Test different task scenarios.
- Check if priorities match expected urgency.
- Verify AI responses follow the required format.

User Testing

- Collect feedback from students/interns.
- Evaluate whether AI suggestions help users decide priorities faster.



Cross Model Comparison

Claude:

- Better at explaining reasoning and asking clarifying questions.
- Generated more detailed architecture discussions.
- Sometimes gave longer answers than needed.

ChatGPT:

- More concise and structured.
- Better at quickly generating implementation-ready formats.
- Sometimes needed more context to understand project decisions.

Final choice:

For planning and exploring ideas, Claude was more useful. For quick implementation structure and coding support, ChatGPT was more efficient.

